Kubernetes cluster or a remote one. We'll also use Octant to examine the remote production cluster where we'll deploy the Polar Bookshop application. For now, close Octant by stopping its process with Ctrl-C.

When you're done, you can stop the Tilt process in the Catalog Service project and run `tilt down` to undeploy the application. Then go to your `polar-deployment` repository, navigate to the kubernetes/platform/development folder, and delete the PostgreSQL installation with `kubectl delete -f services`. Finally, stop the cluster as follows:

```
$ minikube stop --profile polar
```

7.6 Deployment pipeline: Validate Kubernetes manifests

Chapter 3 introduced the concept of a deployment pipeline and its importance in the continuous delivery approach for delivering software quickly, reliably, and safely. So far we've automated the first part of a deployment pipeline: the commit stage. After a developer commits new code to the mainline, this stage goes through build, unit tests, integration tests, static code analysis, and packaging. At the end of this stage, an executable application artifact is published to an artifact repository. That is a *release candidate*.

In this chapter, you learned how to deploy Spring Boot applications on Kubernetes using a declarative approach based on *resource manifests*. They are fundamental to a successful deployment of the release candidate on Kubernetes, so we should guarantee their correctness. This section will show you how to validate Kubernetes manifests as part of the commit stage.